

FPGA based Matrix Multiplication Accelerator

Polash Deb
Ingram School of Engineering
Texas State University
San Marcos, TX, USA
ncf39@txstate.edu

Damian Valles Molina
Ingram School of Engineering
Texas State University
San Marcos, TX, USA
dvalles@txstate.edu

Semih Aslan
Ingram School of Engineering
Texas State University
San Marcos, TX, USA
aslan@txstate.edu

Abstract—The demand for high-speed matrix multiplication continues to grow due to recent developments in images processing, graphics processing, digital signal processing and communication via wireless network. Hence, designing an effective matrix multiplication accelerator is crucial which can perform fast multiplication operation since multiplication on hardware requires much more hardware resources and computational time. To design such kind of hardware, the number of partial products generations must be kept to a minimum, which reduces the amount of add operations and shortens the time it takes to do the multiplication operation. The factor canonical signed digit (FCSD) approach can lower the number of partial products formed during multiplication and eliminate the need for dedicated multiplier. This paper presents a field programmable gate array (FPGA)-based matrix multiplication accelerator that uses FCSD method within a pipeline structure, with a comparative study performed on two distinct Xilinx FPGA platforms: Zynq-7000 and Artix-7.

Keywords— *Matrix Multiplication, CSD, FCSD, FPGA.*

I. INTRODUCTION

Matrix operation plays a major role in many computation-intensive algorithms, including those in neural networks [1], computer vision, numerical modeling, and video processing. Furthermore, it is also frequently utilized in signal processing [2], image-processing for kernel operation [3], computer graphics [4], and quantum mechanics. When a matrix size grows, matrix multiplication becomes a slower process since it needs to perform numerous additions and multiplications [5]. Thus, it is essential to boost the speed of matrix multiplication by using parallel processing and faster multiplication. A pipeline structure for matrix operation enables parallel operation which improves the overall performance, speed, and throughput of a system [6]. Multiplying two numbers can be done more quickly, if there is less generation of partial products. Canonical Sign Digit (CSD) is a recoding method that encodes a number using the fewest possible non-zero digits [7]. Since the number of non-zero bits on the multiplier determines the number of partial products it will produce when multiplying by the multiplicand. Therefore, the number of partial products can be decreased by lowering the number of non-zero bits on the multiplier, which lowers the number of addition operations, and the overall time required to perform a multiplication operation. FCSD is a minor improvement over CSD algorithm which can reduce hardware cost and complexity in filter operation [8][9][10]. FCSD has been applied in different designs to lower filter costs and complexity [8][9][11][12], but its usage remains limited to filter operation.

This study extends the use of FCSD method beyond conventional filter operations to general-purpose matrix multiplication, eliminating DSP blocks entirely and using only LUTs for both multiplication and accumulation.

Furthermore, the design's performance across various FPGA platforms is highlighted by a comparison study on Zynq-7000 and Artix-7. The suggested system can improve overall design performance by utilizing pipeline structures and accelerating multiplication using FCSD algorithms. The proposed system is designed, simulated and synthesized by Xilinx Vivado ML edition 2023.2 [13] tool which employs Verilog language.

The fundamentals of matrix multiplication are covered in Section II. CSD and FCSD techniques are explained in Section III. Section IV presents the proposed design. In the final part, the simulation and synthesis results of the proposed design are shown.

II. MATRIX MULTIPLICATION

A matrix is an organized set of variables or numbers placed in rows and columns. Matrix multiplication involves mathematical operations such as addition, subtraction, and multiplication. Let X be a $a \times b$ matrix where a is the number of rows and b is the number of columns, and let Y be a $b \times c$ matrix where b is the number of rows and c is the number of columns. Then, the multiplication between X and Y matrix generates Z matrix which has a rows and c columns.

$$X = \begin{bmatrix} X_{11} & X_{12} & \dots & X_{1b} \\ X_{21} & X_{22} & \dots & X_{2b} \\ \dots & \dots & \dots & \dots \\ X_{a1} & X_{a2} & \dots & X_{ab} \end{bmatrix} Y = \begin{bmatrix} Y_{11} & Y_{12} & \dots & Y_{1c} \\ Y_{21} & Y_{22} & \dots & Y_{2c} \\ \dots & \dots & \dots & \dots \\ Y_{b1} & Y_{b2} & \dots & Y_{bc} \end{bmatrix} Z = \begin{bmatrix} Z_{11} & Z_{12} & \dots & Z_{1c} \\ Z_{21} & Z_{22} & \dots & Z_{2c} \\ \dots & \dots & \dots & \dots \\ Z_{a1} & Z_{a2} & \dots & Z_{ac} \end{bmatrix} \quad (1)$$

The matrix multiplication $Z=X \times Y$ is usually described by the following equation:

$$Z_{ij} = \sum_{k=0}^{b-1} X_{ik} \times Y_{kj} \quad \text{where } 0 \leq i < a, 0 \leq j < c \quad (2)$$

The simple matrix multiplication algorithm has an $O(abc)$ computational complexity. Because of the cubic complexity, the total number of operations needed increases, which is directly proportional to the product of the matrix dimensions. Therefore, matrix multiplication can become computationally expensive for large matrices.

III. OVERVIEW OF CSD AND FCSD

The CSD representation is a signed digit (SD) format with unique characteristics ideal for applications requiring very quick calculation, low power consumption, and efficient area [14]. The CSD is constructed by the ternary number system (0,1 and -1). In CSD, the number of nonzero digits is minimum and there are no adjacent nonzero digits. Therefore, multiplication is one of the primary applications for CSD numbers, since it minimizes the partial product and produces the multiplication's product with the fewest possible adders and subtractors. According to an earlier study, using CSD the amount of partial product additions can be reduced by approximately 33% [7]. The mathematical formula for representing an unsigned binary number is given below.

$$A = \sum_{k=0}^{N-1} a_k 2^k \quad \text{where } a_k \in \{0,1\} \quad (3)$$

B is the $(N+1)$ digit CSD representation of A defined by

$$B = \sum_{k=0}^{N-1} a_k 2^k = \sum_{k=0}^N b_k 2^k \quad (4)$$

where $b_k \in \{0,1,-1\}$

Since there should not be any adjacent nonzero digits in CSD number which can be represented by

$$B_{k+1} B_k = 0 \quad \text{where } 0 \leq k \leq N-1 \quad (5)$$

The probability of nonzero digit present in a N digit CSD number [15] is shown by.

$$P(|b_k| = 1) = \frac{1}{3} + \frac{1}{9N} \left[1 - \left(\frac{-1}{2} \right)^N \right] \quad (6)$$

By analyzing adjacent digit pairs of a binary number from the LSB end to the MSB+1 end, CSD encoding can be achieved. Since the nearby digits pairs must be checked to generate a CSD number, an additional 0 digit is required in MSB+1 position to obtain a CSD number that is the same length as the original binary number.

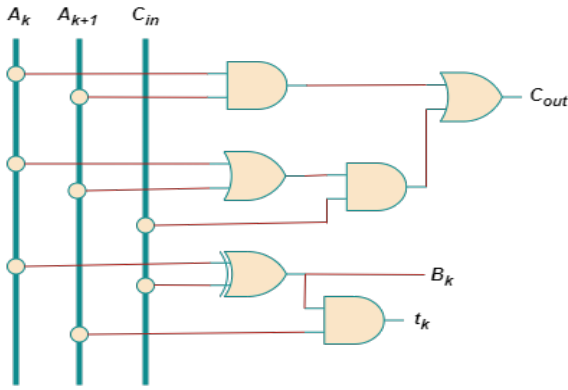


Fig. 1. Binary to CSD conversion circuit.

In figure 1, the neighboring digits of the binary number are A_k and A_{k+1} and the output CSD digit is B_k . C_{in} and C_{out} represent carry in and carry out respectively. The sign of CSD digit is determined by t_k . FCSD replaces multiplication with add, subtract, and shift operations. Accurate factoring and CSD encoding of a number together result in fewer adders in multiplication with another number, and which further lowers hardware requirements [9]. The implementation of FCSD in fir filter is highly effective [8] [9] [10].

IV. PROPOSED DESIGN

In this work, a pipeline architecture is used which has six different stages for performing specific tasks. Figure 3 represents the proposed pipeline structure. The first stage of pipeline architecture stores the input matrices into registers to perform different operations into later stages. During the first clock cycle, the first row of the first input matrix and the first column of the second input matrix are stored. The first row of the first input matrix and the second column of the second input matrix are stored in the following clock. Storing the first row of the first input matrix continues until the entire second matrix is stored. This process repeats for the rest of the row of

the first input matrix until the first input matrix is fully stored. Only one row and one column are stored in each clock cycle. In the subsequent clock cycle, a new input replaces the stored data. It is hence area efficient. Figure 2 shows the direction of storing two input matrices.

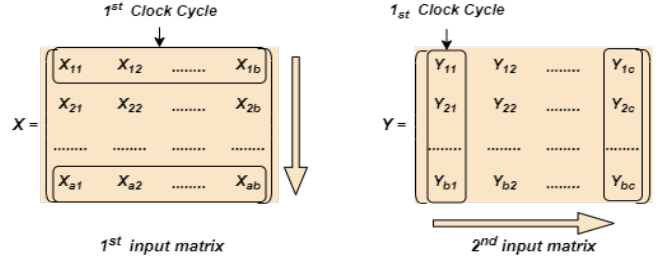


Fig. 2. Storing process of two input matrices.

All stages process data that comes from the stage before it. In the second stage of pipeline, each element in each column of the second input matrix is split into two factors, each of which is stored in different registers. In the following step, those factors are converted into equivalent CSD numbers, which are similarly kept in separate registers. The fourth stage involves multiplying the first column of CSD numbers by the row of the first input matrix. The outcome of the fourth stage is multiplied by the CSD numbers in the second column in the fifth stage. In the sixth or final step, accumulation takes place. Each stage in the pipeline depends on the stage before it and completes an operation in single clock cycle. If a stage halts activity, the next stage will cease operations after one clock cycle. The first stage remains active until finish reading the entire two input matrices. The initial output is obtained after 5 clock cycles delay because of the six pipeline stages. Therefore, the sixth stage quits functioning 5 clock cycles after the first stage inactivity. The latency of this pipeline is 5 clock cycles. If two $N \times N$ input matrices are considered, then the total number of clock cycles needed for computing the resultant matrix can be calculated. Since the pipeline stores one row and one column at a time and this procedure continues. Therefore, it takes N clock cycles to compute the first row of the resultant matrix. However, because of the 5 clock cycles delay in the pipeline, it takes an additional 5 clock cycles for the result of the first row to be available at the output. The first row of the resultant matrix is entirely computed after $T_1 = N + 5$ clock cycles. The same procedure is repeated for the second row of the resultant matrix and the second row is generated after another N clock cycles. Therefore, total clock cycle requires for generating first and second row of output matrix is $T_2 = 2N + 5$. Likewise, the total time required to calculate first, second and third rows of output matrix is $T_3 = 3N + 5$. This pattern is repeated for all N rows; hence the full resulting matrix is computed after $T_{total} = N^2 + 5$ clock cycles. Figure 4 illustrates the internal structure of proposed pipeline for generating the output matrix when both input matrices size is $N \times N$. It also shows how the pipeline stores a row and a column in single clock cycle, processes them, transfer one stage output to another stage, and produces an element of the output matrix. $X_{11} \dots X_{1N}$ are values of the first row of the first input matrix, $Y_{11} \dots Y_{N1}$ are elements of the first column of the second input matrix and R_n, R_{n+1} represent registers for storing numbers.

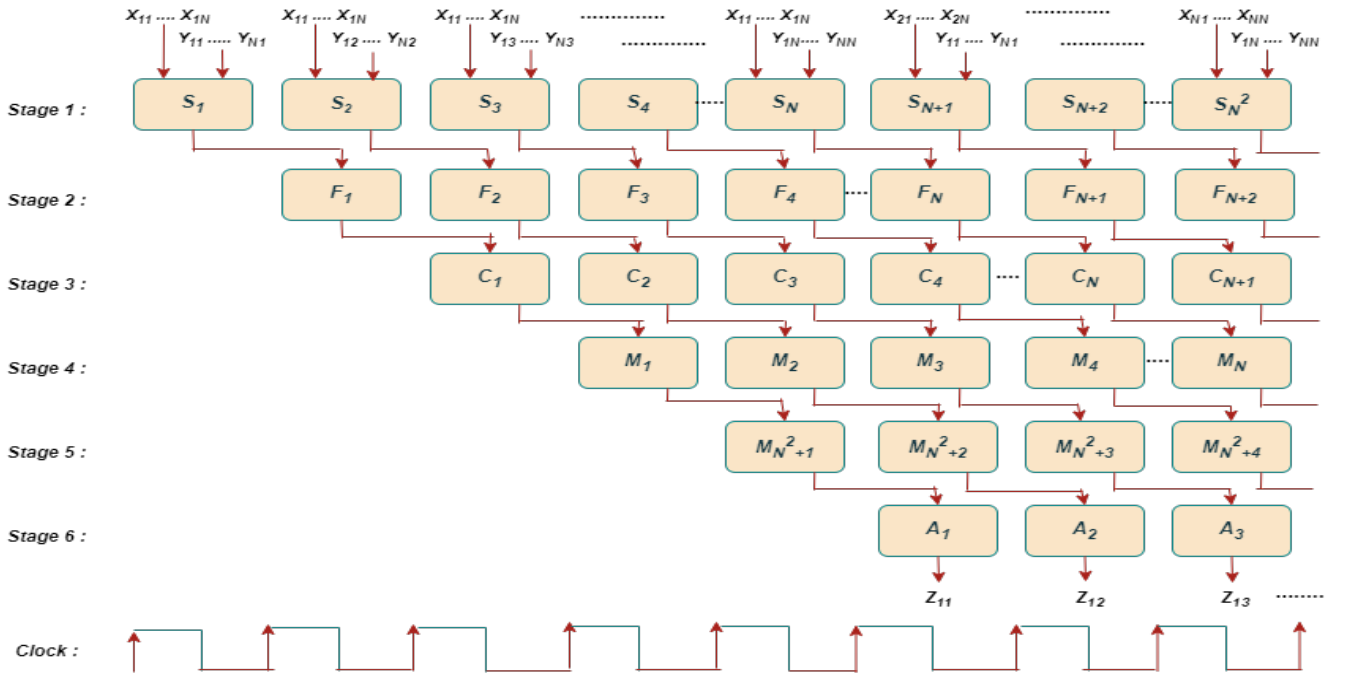


Fig. 3. Proposed pipeline architecture.

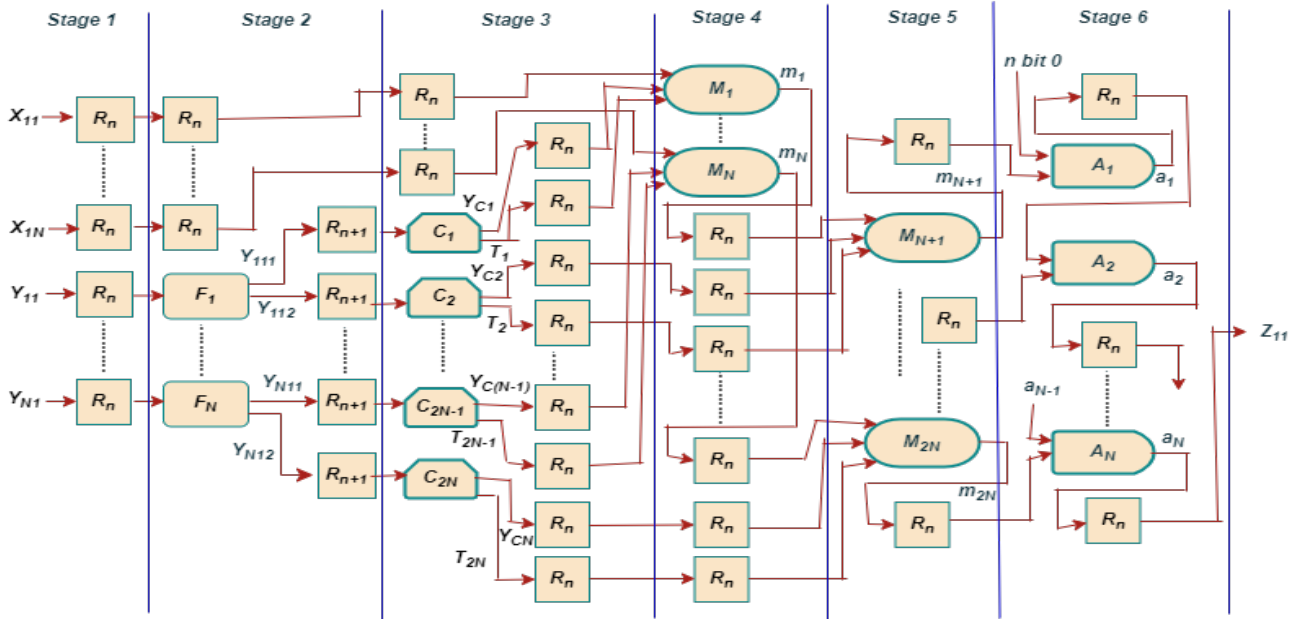


Fig. 4. Internal structure of proposed pipeline.

A. CSD Module

Figure 5 exhibits the process of converting an ordinary binary number to a CSD number where the digits a_k and a_{k+1} are neighboring digits of the binary number, b_k is the CSD digit and n is the number of bits present in the input binary number. In CSD method, the neighboring digit pairs from the LSB to the MSB+1 are analyzed and MSB+1 bit is set 0 for both positive and negative number. Since the CSD has an additional digit -1, it differs from a traditional binary number. Therefore, it is crucial to keep track of where the bit position of -1 is in CSD number. t_k keeps track of -1. If there is -1 in CSD number, then t_k is set to 1. However, for 0 and 1, it is set to 0.

B. Multiplication and Addition Module

In digital hardware, multiplication is typically more expensive and takes longer to finish than other operations. Shift and add operation are another way to represent multiplication in hardware which is more resource efficient than dedicated multiplication units. Figure 6 shows multiplication operation which is replaced by add, subtract, and shift operation. n is the number of bits present in the input CSD number. Depending on t_k , add and subtraction operation are performed. If t_k is 1, which indicates that there is -1 in k th bit position of b , then subtraction operation is performed with accumulator after a is shifted k bits left. Otherwise, add operation is carried out depending on b_k value with accumulator after doing left shifting. Figure 7 displays the process of adding two fixed point numbers in the

proposed design. Subtraction and addition are carried out based on the sign bits of both numbers. 2's Complement answer can be avoided with this procedure.

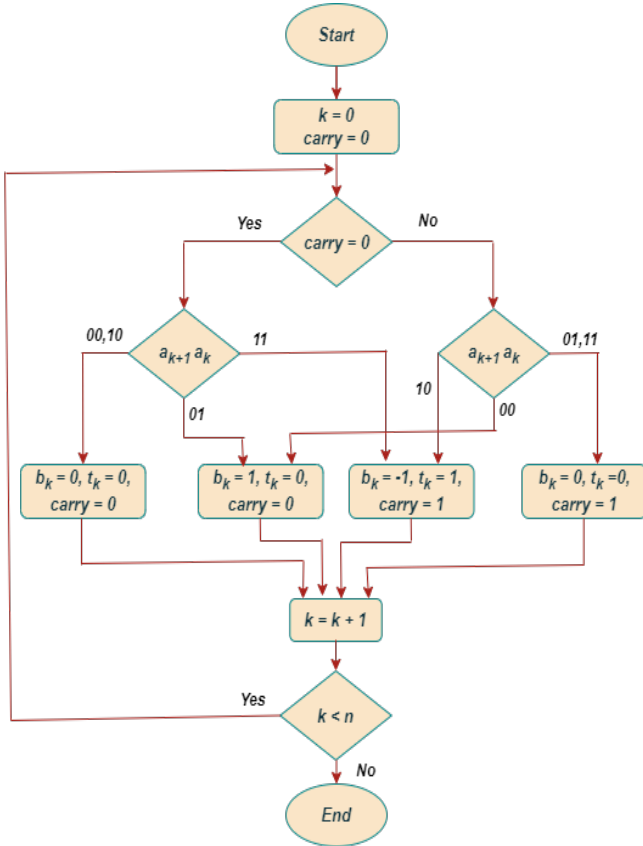


Fig. 5. Conversion of a binary number to CSD.

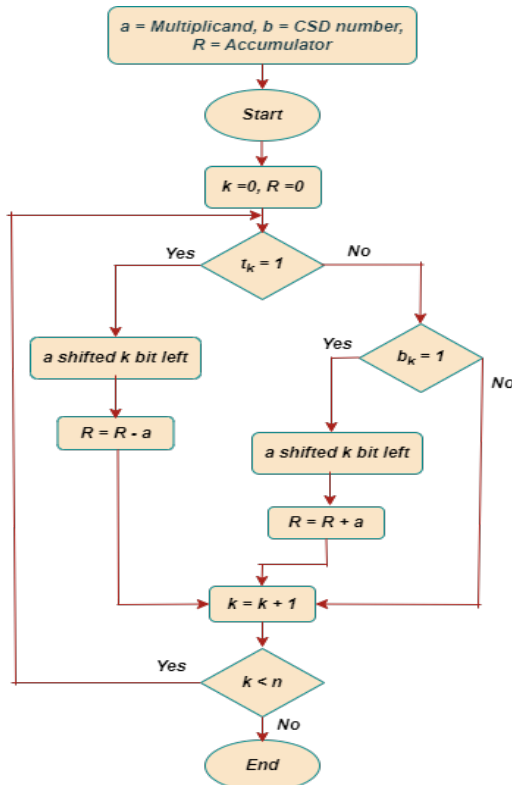


Fig. 6. Multiplication process between a CSD and binary number.

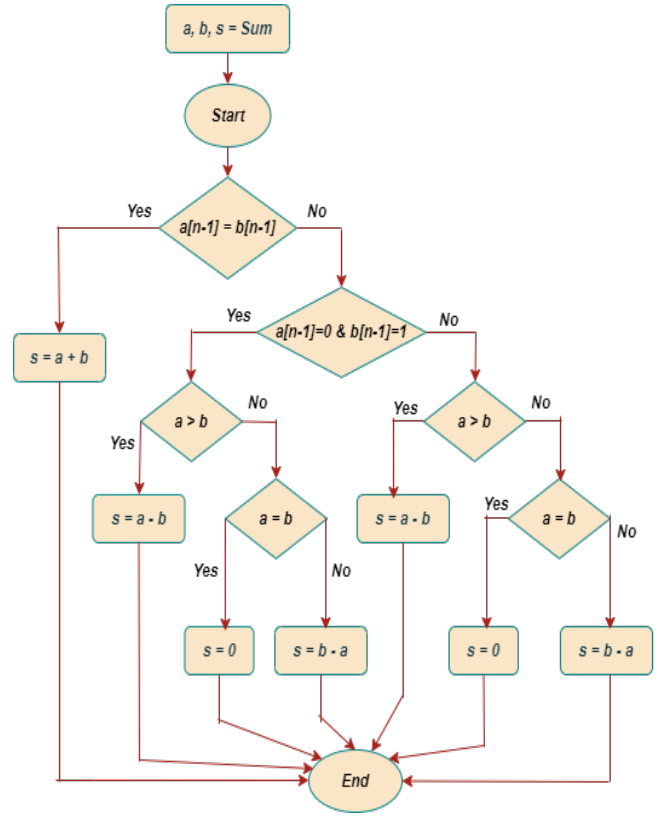


Fig. 7. Addition process of two fixed point numbers.

C. Area

If two $N \times N$ matrices are multiplied, then the number of multiplications and add operation needed to compute the output matrix can be determined.

$$\text{No of multiplication} = (NM + NM)N^2 = 2N^3M \quad (7)$$

$$\text{No of addition} = N^3A \quad (8)$$

This proposed design requires more number multiplication. Since, to apply the FCSD method, each element of the second input matrix is split into two factors, which are then multiplied in two different stages of the pipeline. Estimated area can be found by considering pipeline (P), factor module (F), CSD module (C), multiplication module (M) and addition module (A) together.

$$\begin{aligned} \text{Estimated area} &= P + F + C + M + A \\ &= \{P_1 + P_2 + P_3 + P_4 + P_5 + P_6\} + F + C + M + A \end{aligned}$$

$$\begin{aligned} &= \{(6R_n + 2NR_n) + (NR_n + 4NR_{n+1}) + (5NR_n + 4NR_{n-q-1}) + (3NR_n + NR_{n-q-1}) + 2NR_n + (N+1)R_n\} \\ &\quad + 2NR_{n+1} + NR_{n-q-1} + (2NR_{2n-2} + 20NA_{2n-2}) + (NR_n + NA_{n-1}) \\ &= (15N + 7)R_n + 6NR_{n+1} + 6NR_{n-q-1} + 2NR_{2n-2} + NA_{n-1} + 20NA_{2n-2} \end{aligned} \quad (9)$$

Where,
 n : input bits.
 q : fraction bits.

R_n : n bit Register.
 R_{n+1} : $(n + 1)$ bit Register.
 R_{2n-2} : $(2n - 2)$ bit Register.
 R_{n-q-1} : $(n - q - 1)$ bit Register.
 A_{n-1} : $(n - 1)$ bit Adder.
 A_{2n-2} : $(2n - 2)$ bit Adder.

V. RESULT AND DISCUSSION

This study uses the estimated clock frequency that is obtained after the synthesis process on Zynq 7000 ZC702 and Artix-7 AC701 to determine the throughput. When the size of the matrix grows in an FPGA, the computing constraints make a nearly fixed throughput on the operations. The throughput can be determined by dividing the total number of fixed-point operations required for matrix computation by the execution time [16]. Throughput (in Giga operation per seconds (GOP/s)) comparison between two FPGA architectures is plotted in figure 8 with $N \times N$ configuration where $N = 2, 3, 4, \dots$ means 2×2 , 3×3 matrix and so on. Both FPGAs exhibit an increase in throughput as matrix size grows because the design becomes more computationally intensive and makes better use of hardware resources. After 34×34 matrix size, throughput saturates on Zynq-7000. This is because of limitations in hardware resources, particularly a lack of LUT resources resulting in a peak throughput of 4.23 GOP/s. On the other hand, in Artix-7, the throughput keeps increasing until the matrix size reaches 71×71 , at which point it reaches saturation and achieves a peak throughput of 11.11 GOP/s. This is because it has more resources available and lower clock period.

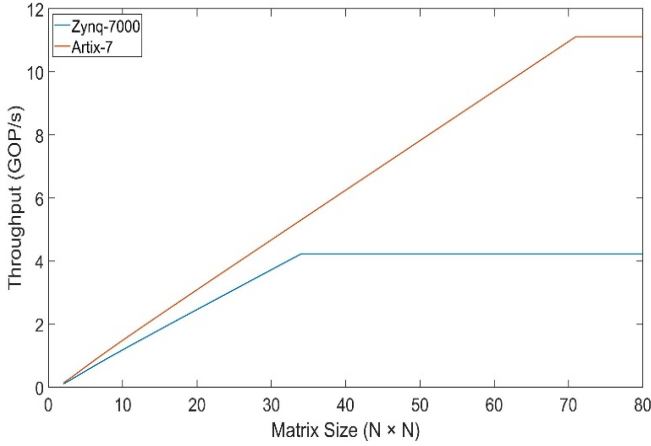


Fig. 8. Performance comparison of the proposed design on Zynq-7000 and Artix-7 FPGAs.

The execution time (in seconds) of the proposed design is examined on both Artix-7 and Zynq-7000 FPGAs demonstrate in figure 9. The largest matrix size supported by Zynq-7000 is 34×34 due to resources restrictions. So, in order to keep things fair, the execution time comparison is limited to matrix sizes up to 34×34 . The proposed architecture in Artix-7 has a consistently faster execution time across all matrix sizes within the common range, due to its lower clock period.

Table I presents an analysis of the area in terms of LUTs and slice registers across various matrix sizes on both FPGA platforms. The findings demonstrate that when matrix size grows, both FPGA exhibit similar trends in area consumption. However, the Artix-7 can support bigger

matrix sizes than the Zynq-7000 because of higher LUT capacity. Slice register utilization remains within bounds on both devices for all matrix sizes. Since the design does not use embedded DSP blocks for multiplication or addition, LUT capacity is the primary factor which determines the maximum matrix size an FPGA can support.

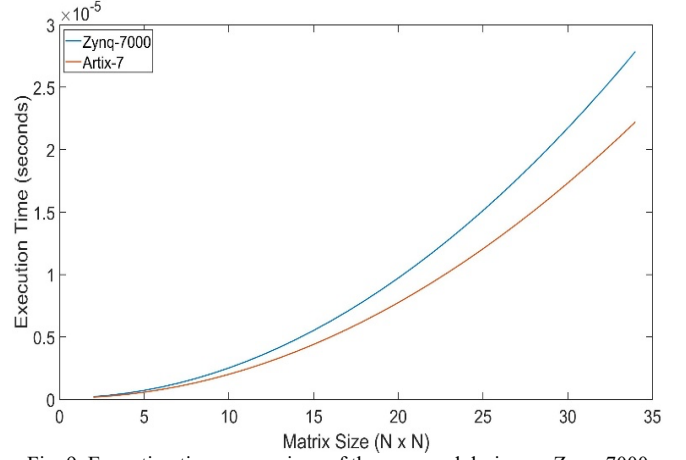


Fig. 9. Execution time comparison of the proposed design on Zynq-7000 and Artix-7 FPGAs.

Table I. Area of the proposed design with different matrix sizes on Zynq-7000 and Artix-7 FPGAs.

Matrix Size	Zynq-7000 ZC702		Artix-7 AC701	
	LUTs (53200)	Slice Register (106400)	LUTs (134600)	Slice Register (269200)
4×4	5042	1014	5042	1014
10×10	13177	2434	13190	2441
30×30	45651	7273	45675	7286
34×34	51751	8168	51805	8185
50×50	Over limit	11931	83586	11947
60×60	Over limit	14297	106662	14297
70×70	Over limit	16669	128993	16677
71×71	Over limit	16916	130777	16916

Figure 10 exhibits the results of a behavioral simulation run on two 4×4 input matrices, where each element of the input and output matrices is represented by 16 bits: first bit (MSB) for the sign, the next 10 bits for the integer part, and final 5 bits for the fraction part.

VI. CONCLUSION

This work presents an FPGA-based matrix multiplication accelerator that uses the FCSD method, substitutes conventional DSP-based operations with LUT-based operations. In order to evaluate performance, the design is synthesized on two different FPGA devices. According to the results, LUT utilization is the limiting factor, particularly on the Zynq-7000, even though both systems maintained acceptable slice register usage. The Artix-7 can handle larger matrices, which makes it more appropriate for a bigger design. The results of this work demonstrate that the proposed design performs better when implemented on FPGAs with more hardware resources (e.g., LUTs). The proposed design provides a strong substitute for matrix multiplication on platforms where dedicated DSP block is limited. Although this design avoids DSPs and works well on devices with

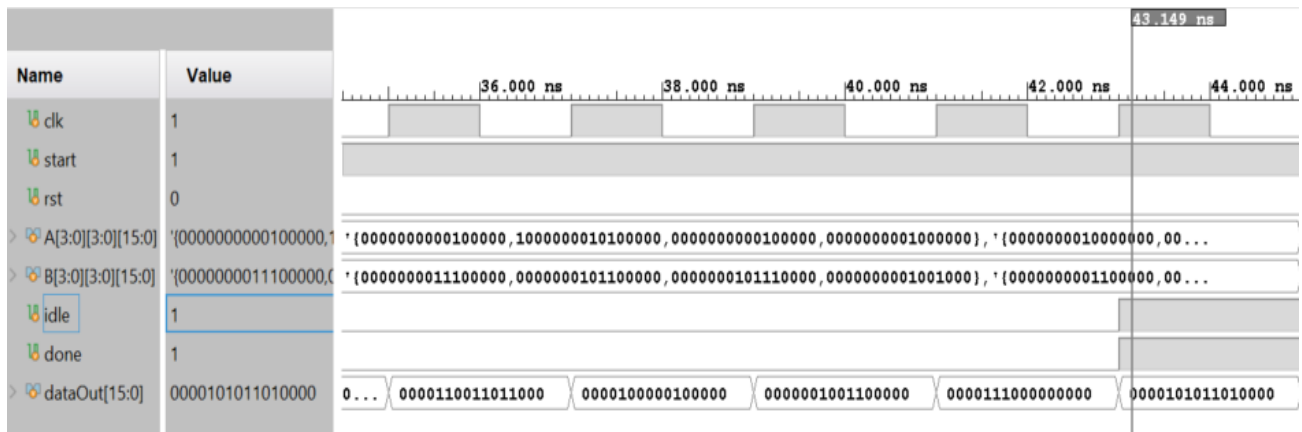


Fig. 10. Simulation result of 4×4 matrix multiplication.

many LUTs, the design scalability for bigger matrices is limited by its quickly increasing LUT utilization, particularly on FPGAs with limited resources. Hybrid FCSD-DSP designs may be investigated in future research to enhance area and performance and assess the design on a wider variety of FPGAs, including high-end ones.

REFERENCES

- [1] J. V. Frances-Villora, A. Rosado-Muñoz, J. M. Martínez-Villena, M. Bataller-Mompean, J. Fco. Guerrero, and M. Wegrzyn, "Hardware implementation of real-time Extreme Learning Machine in FPGA: Analysis of precision, resource occupation and performance," *Comput. Electr. Eng.*, vol. 51, pp. 139–156, Apr. 2016, doi: 10.1016/j.compeleceng.2016.02.007.
- [2] A. Amira, A. Bouridane, and P. Milligan, "Accelerating Matrix Product on Reconfigurable Hardware for Signal Processing," in *Field-Programmable Logic and Applications*, G. Brebner and R. Woods, Eds., in Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2001, pp. 101–111. doi: 10.1007/3-540-44687-7_11.
- [3] S. M. Qasim, A. A. Telba, and A. Y. AlMazroo, "FPGA design and implementation of matrix multiplier architectures for image and signal processing applications," *Int. J. Comput. Sci. Netw. Secur.*, vol. 10, no. 2, pp. 168–176, 2010.
- [4] F. Bensaali, A. Amira, and A. Bouridane, "Accelerating matrix product on reconfigurable hardware for image processing applications," *IEE Proc. - Circuits Devices Syst.*, vol. 152, no. 3, pp. 236–246, Jun. 2005, doi: 10.1049/ip-cds:20040838.
- [5] Y. Tan, T. Imamura, and D. Mukunoki, "Design of an FPGA-Based Matrix Multiplier with Task Parallelism," in *Parallel Computing: Technology Trends*, IOS Press, 2020, pp. 241–250. doi: 10.3233/APC200047.
- [6] S. Aslan, C. Desmouliers, E. Oruklu, and J. Saniie, "High performance reconfigurable pipelined matrix multiplication module designer," in *2010 IEEE International Conference on Electro/Information Technology*, May 2010, pp. 1–6. doi: 10.1109/EIT.2010.5612172.
- [7] R. M. Hewlitt and E. S. Swartzlantler, "Canonical signed digit representation for FIR digital filters," in *2000 IEEE Workshop on SiGNAL PROCESSING SYSTEMS. SiPS 2000. Design and Implementation (Cat. No.00TH8528)*, Oct. 2000, pp. 416–426. doi: 10.1109/SIPS.2000.886740.
- [8] N. Goel and A. Nandi, "Design of FIR Filter Using FCSD Representation," in *2015 IEEE International Conference on Computational Intelligence & Communication Technology*, Feb. 2015, pp. 617–620. doi: 10.1109/CICT.2015.85.
- [9] R. Lal, R. Mehra, and S. S., "FPGA based Band Pass FIR Filter using Factored Canonic Signed Digit Technique for Satellite Application," *Int. J. Comput. Appl.*, vol. 156, no. 3, pp. 45–49, Dec. 2016, doi: 10.5120/ijca2016912408.
- [10] K. Priya and R. Mehra, "FPGA based cost efficient FIR filter using Factored CSD technique," *Int. J. Recent Trends Eng.*, vol. 1, no. 6, 2013.
- [11] G. S. Gawande and K. B. Khanchandani, "Efficient Design and FPGA Implementation of Digital Filter for Audio Application," in *2015 International Conference on Computing Communication Control and Automation*, Feb. 2015, pp. 906–910. doi: 10.1109/ICCUBEA.2015.180.
- [12] I. Sharma, A. Kumar, G. K. Singh, and H.-N. Lee, "Design of multiplierless prototype filter for two-channel filter bank using hybrid method in FCSD space," *IET Circuits Devices Syst.*, vol. 11, no. 1, pp. 29–40, 2017, doi: 10.1049/iet-cds.2016.0124.
- [13] "Vivado Overview," AMD. Accessed: Jan. 28, 2024. [Online]. Available: <https://www.xilinx.com/products/designtools/vivado.html>
- [14] I. Koren, *Computer Arithmetic Algorithms*, 2nd edition. Natick, MA: A K Peters/CRC Press, 2001.
- [15] G.-K. Ma and F. J. Taylor, "Multiplier policies for digital signal processing," *IEEE ASSP Mag.*, vol. 7, no. 1, pp. 6–20, Jan. 1990, doi:10.1109/53.45968.
- [16] L. Song, Y. Chi, A. Sohrabizadeh, Y. Choi, J. Lau, and J. Cong, "Sextans: A Streaming Accelerator for General-Purpose Sparse-Matrix Dense-Matrix Multiplication," in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, in FPGA '22. New York, NY, USA: Association for Computing Machinery, Feb. 2022, pp. 65–77. doi: 10.1145/3490422.3502357.